ARCHITECTURAL SPECIFICATION & ROADMAP

Smart Fuel Management System

Technical Development Roadmap for Incremental System Enhancement

Author: Senior Software Architect

Domain: Industrial IoT, Microservices, Redis Caching, Secure Authentication & AI Integration

Target System: Java & Spring Boot Enterprise Architecture

Version: 2.0-PROD-REF

Date: October 2026

1. Executive Summary

The Smart Fuel Management System is an active, production-ready enterprise solution built on a Java and Spring Boot microservices architecture. It coordinates vehicle identification, session execution, wallet transactions, payment gateways, and pump telemetry across distributed fuel stations. To scale performance, ensure financial auditability, upgrade security protocols, and leverage industrial telemetry, this document outlines a rigorous, phased technical development roadmap.

> **Incremental Architecture Mandate:** The existing software is fully operational. This initiative represents an incremental engineering roadmap rather than a ground-up rewrite. Core microservices, database schemas, and message broker paradigms are preserved. Enhancements are introduced cleanly via modular service extensions, asynchronous event-driven telemetry pipelines, and targeted Redis caching layers.

2. Table of Contents

- **Section 1:** Executive Summary & Scope
- **Section 2:** Existing System Context & Architecture Baseline
- **Section 3:** Phase One — Distributed Caching Implementation
- **Section 4:** Phase Two — Physical Fuel-Storage Inventory Monitoring
- **Section 5:** Phase Three — Secure Credential-Card Authentication Migration
- **Section 6:** Phase Four — Future AI Capabilities & Advisory Analytics
- **Section 7:** Overall Implementation Order & Dependencies
- **Section 8:** Technical Architecture & Data Flow Diagrams
- **Section 9:** Practical Deliverables, Backlogs & Definition of Done
- **Section 10:** Technical Glossary & Compliance References

3. Existing System Context & Architecture Baseline

Before introducing new functional capabilities, establishing an unshakeable inventory of current architectural assets, known technologies, and unverified boundaries is critical. Distinguishing verified baseline components from assumptions prevents regressions in production.

Subsystem / Domain	Known Architecture & Technologies	Status / Verification Required
Core Platform	Java, Spring Boot, Maven, Spring Cloud Gateway, Eureka Service Discovery.	**Verified** Production Active
Security & Auth	JWT authentication, role-based authorization filters across gateway.	**Verified** Production Active
Persistence	MySQL relational databases per microservice domain.	**Verified** Production Active
Real-Time State	Redis for pump heartbeat tracking (`pump:heartbeat:*`, TTL ~45s). Pump states: ONLINE, FUELING, PAUSED, OFFLINE, ERROR.	**Verified** Production Active
Messaging	MQTT communication configured with localhost broker and QoS 1.	**Inspection** Broker topology & scaling
Core Services	Users, Authentication, Wallets, Payments, Transactions, Payment Methods, Fuel Sessions.	**Verified** Production Active
Identification	Camera license-plate recognition (LPR) and basic RFID-tag identification.	**Inspection** Hardware reader protocols

Critical Architectural Boundary: Financial transactions, wallet balances, and fuel authorization decisions are strictly transactional and isolated within MySQL database engines. Redis is utilized exclusively for ephemeral state tracking (heartbeats). Caching layers introduced in Phase 1 must strictly adhere to this separation of concerns.

4. Phase One: Implement Caching Across Microservices

To reduce database load and improve response latencies for read-heavy operations, Phase One introduces a structured caching strategy across selected microservices utilizing Spring Cache abstraction backed by Redis.

Key Design Principles & Governance

- **Target Data Selection:** Restrict caching strictly to read-heavy, low-volatility reference data: station configurations, pump metadata, fuel grade definitions, and static pricing rules.
- **Strict Financial Exclusion:** *Never* cache wallet balances, payment status tokens, transaction history, or security-critical fuel authorization flags. Stale financial data introduces severe operational and monetary risk.
- **Namespace & Naming Convention:** Cache keys must follow strict hierarchical naming: `[service]:[entity]:[id]` (e.g., `station:config:station_902`).
- **Invalidation Strategy:** Implement cache-aside or write-through eviction policies triggered by administrative update events published via Spring Application Events or Redis Pub/Sub.

Cache Region	Target Data	Default TTL	Invalidation Trigger
`station:config`	Station metadata, operating hours, active nozzles	15 Minutes	Admin configuration update API call
`pump:metadata`	Pump hardware mappings, fuel grade allocation	30 Minutes	Hardware reconfiguration event
`ref:fuel-grades`	Global fuel type definitions and chemical properties	24 Hours	Manual administrative refresh

> **Resiliency & Fallback:** In the event of a Redis cluster outage, caching interceptors must catch `RedisConnectionException`, log a warning metric, and transparently fall back to querying the MySQL database without failing the user request.

5. Phase Two: Physical Fuel-Storage Inventory Monitoring

Phase Two bridges the gap between software-recorded sales and physical fuel storage through automated IoT tank monitoring. This ensures complete accounting reconciliation across retail stations.

Complete Data Flow Architecture

`Physical Tank → Magnetostrictive / Radar Sensor → Sensor Controller (PLC/RTU) → TLS / Authenticated MQTT Broker → Backend Telemetry Ingestion Service → MySQL Inventory Ledger → Real-Time Admin Dashboard & Alerts`

Hardware Selection & Measurement Considerations

- **Sensor Technologies:** Magnetostrictive probes provide high-precision level and temperature measurement; radar sensors offer non-contact measurement ideal for corrosive or high-viscosity bio-fuels.
- **Tank Geometry & Calibration:** Cylindrical or horizontal underground tanks require accurate strapping (calibration) tables mapping raw height (mm) to calibrated volume (liters), accounting for thermal expansion.
- **Intrinsic Safety:** All hardware deployed in hazardous Class I, Div 1 / Zone 0 environments storing flammable gasoline must be intrinsically safe (ATEX / IECEx certified) with galvanic isolators.
- **Sensor Health & Stale Detection:** Telemetry messages must include UNIX timestamps. If no valid packet is received within a configurable window (e.g., 300 seconds), flag sensor connectivity as `OFFLINE` and trigger an alert.

> **Service Architecture Decision:** Introduce a dedicated `inventory-service` microservice to encapsulate tank registries, telemetry processing, and reconciliation ledgers, maintaining clean bounded contexts separate from transaction and payment services.

Inventory Calculations & Reconciliation Logic

Inventory reconciliation relies on rigorous accounting formulas comparing physical sensor observations against expected software ledgers:

```
Expected Stock = Opening Stock + Deliveries − Recorded Sales + Authorized
Adjustments
Inventory Discrepancy = Measured Stock (Sensor) − Expected Stock
```

Operational Rule: A negative discrepancy indicates potential loss, evaporation, metering error, or unrecorded leakage. It is *not* conclusive proof of theft and must trigger an administrative audit workflow rather than automated punitive actions.

Tank Dashboard Metrics & Illustrative Data

Parameter	Tank 01 (Diesel)	Tank 02 (Unleaded 95)
Total Capacity	50,000 Liters	30,000 Liters
Current Measured	38,450 Liters	18,120 Liters
Capacity %	76.9%	60.4%
Expected Quantity	38,600 Liters	18,050 Liters
Current Discrepancy	-150 Liters (-0.38%)	+70 Liters (+0.39%)
Sensor Health	ONLINE (Last: 12s ago)	ONLINE (Last: 8s ago)

6. Phase Three: Secure Credential-Card Authentication

Phase Three replaces legacy camera LPR and basic static RFID identification with secure, cryptographically verified NFC smart cards, mitigating credential cloning and unauthorized vehicle refueling.

Security & Cryptographic Architecture

- **Smart Card Selection:** Utilize MIFARE Desfire EV3 or ISO/IEC 14443-4 compliant smart cards supporting AES-128 cryptographic challenge-response authentication. Static UID-only cards are strictly prohibited due to cloning vulnerabilities.
- **Four-Tier Security Separation:**
  1. **Credential Authentication:** Proving cryptographic possession of authorized smart card keys.
  2. **Vehicle Verification:** Establishing that the authorized vehicle is physically present at the fueling bay.
  3. **Fuel Authorization:** Backend evaluation of limits, wallets, and operational status.
  4. انب**Physical Dispensing Control:** Pump controller hardware enforcing the authorized session window.
- **Replay & Clone Protection:** Implement rolling transaction nonces and timestamp verification at the station gateway to prevent replay attacks.
- **Offline Resiliency:** Station gateways maintain cached signed authorization tokens with short TTLs (e.g., 120 seconds) to ensure reliable operation during intermittent WAN outages.

> **Inspection Prerequisite:** Inspect existing physical reader hardware, serial/TCP communication protocols between pumps and gateway controllers, and gateway firmware capabilities before purchasing smart card inventory.

7. Phase Four: Future AI Capabilities & Advisory Analytics

Once reliable transaction logs, IoT tank telemetry, and secure session credentials are fully established, Phase Four introduces intelligent advisory AI models to optimize operations without compromising system safety.

Proposed AI Modules & Operational Scope

- **Fuel Consumption Anomaly Detection:** Statistical baseline models and Isolation Forests flagging abnormal fuel extraction rates per session or vehicle.
- **Tank Discrepancy & Leak Detection:** Time-series regression analyzing cumulative sensor drift versus recorded sales to isolate potential underground pipe leaks or meter calibration errors.
- **Pump Failure Prediction:** Predictive maintenance models evaluating pump heartbeat jitter, error frequency, and motor runtime telemetry.
- **Demand Forecasting:** ARIMA / Prophet time-series models predicting station fuel replenishment schedules based on historical traffic.

> **Strict Safety Mandate:** AI modules operate in an advisory capacity only. *Never* permit AI systems to independently authorize fuel dispensing, modify wallet balances, alter financial records, or override hardware safety interlocks. An AI anomaly flag requires human investigation and is never proof of fraud.

Graduated Implementation Approach

Initiate Phase Four with explainable rule-based algorithms and statistical standard-deviation baselines. Transition to complex machine learning models only after achieving a minimum of 12 months of clean, verified telemetry and transactional data.

8. Overall Implementation Order & Phase Dependencies

Executing this roadmap requires strict adherence to sequential dependencies, ensuring foundational system stability and telemetry reliability before deploying high-risk physical hardware or advisory AI engines.

Recommended Execution Sequence

1. **Phase 1 (Caching):** Low risk, high reward. Optimizes existing microservice performance and establishes Redis governance patterns. (Estimated Effort: Medium)
2. **Phase 2 (IoT Inventory):** Hardware procurement must initiate immediately in parallel with Phase 1 due to long lead times. Establishes physical telemetry pipelines. (Estimated Effort: Large)
3. **Phase 3 (Credential Cards):** Replaces identification security layers. Requires gateway firmware inspection and hardware retrofits across stations. (Estimated Effort: Large)
4. **Phase 4 (AI Capabilities):** Dependent on mature, clean datasets generated by Phases 1 through 3. (Estimated Effort: Medium)

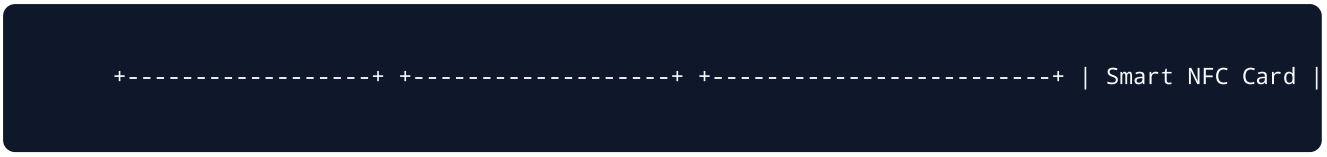
> **Cross-Cutting Requirements:** Every phase must incorporate robust security reviews, comprehensive Prometheus/Grafana observability, structured audit logging, idempotent API design, and automated regression testing suites.

9. Technical Architecture & Data Flow Diagrams

Visual representation of the physical sensor-to-MQTT-to-backend inventory pipeline:



Visual representation of the secure credential-card authentication flow:



10. Practical Deliverables & Master Backlog

A prioritized task backlog ensures clear accountability and tracks completion criteria across all project phases.

Task ID	Phase	Task Description	Priority	Effort
TSK-01	Phase 1	Audit existing Redis configuration and memory limits	High	Small
TSK-02	Phase 1	Implement Spring Cache with Redis for station config	High	Medium
TSK-03	Phase 2	Procure and bench-test industrial tank level sensors	Critical	Large
TSK-04	Phase 2	Develop `inventory-service` and MQTT ingestion consumer	High	Large
TSK-05	Phase 3	Evaluate NFC smart card readers and DESFire keys	High	Medium
TSK-06	Phase 3	Implement cryptographic challenge-response validation	Critical	Large
TSK-07	Phase 4	Build statistical fuel anomaly detection baseline	Medium	Medium

Definition of Done (DoD)

- Code successfully compiled, unit-tested (>85% coverage), and integrated into Maven multi-module builds.
- Executed automated integration tests verifying idempotency and Redis failure fallback.
- Peer reviewed by Senior Architect and deployed to staging environment with zero regressions.

11. Technical Glossary & Next Steps

Glossary of Technical Terms

- **Magnetostrictive Probe:** High-precision industrial sensor measuring liquid level and temperature via magnetostrictive delay lines.
- **QoS 1:** MQTT Quality of Service level ensuring delivery of messages at least once between broker and clients.
- **Cache-Aside:** Caching pattern where application code manages cache population, checking cache before querying database.
- **Strap Table:** Calibration table converting measured liquid height into exact volumetric quantities for irregularly shaped tanks.
- **DESFire EV3:** High-security NFC smart card chip supporting AES hardware encryption and secure messaging.

12. Immediate Next Steps

1. Conduct comprehensive repository and codebase review to map out existing service boundaries and endpoints.
2. Establish baseline performance metrics for database query latencies prior to Phase 1 caching deployment.
3. Initiate hardware vendor outreach for physical tank level sensors and NFC readers.